

DSA Assignment : 2

Submitted by: Tushar Saini

Roll no. 2501730053

B.TECH CSE(AI&ML)

Section : A

Submitted to: Mrs. Neetu Chauhan

□ 1. Amortized Analysis of Dynamic Array

In a dynamic array, elements are added using the `append()` operation. Most of the time, adding an element takes **constant time $O(1)$** because the element is simply placed at the next available position.

However, when the array becomes full, it needs to **resize**. During resizing:

- A new array of double capacity is created
- All existing elements are copied to the new array
- This process takes **$O(n)$** time

Even though resizing is expensive, it does not happen frequently. Therefore:

- Most operations are fast ($O(1)$)
- Only a few operations are slow ($O(n)$)

□ So, the **overall average time complexity is $O(1)$** , which is called

amortized time complexity.

☐ 2. Stack Complexity

A **Stack** follows the **LIFO (Last In First Out)** principle, meaning the last element added is the first one removed.

Operations:

- **push(x):** Adds an element to the top → **O(1)**
- **pop():** Removes the top element → **O(1)**
- **peek():** Returns the top element without removing → **O(1)**

☐ 3. Queue Complexity

A **Queue** follows the **FIFO (First In First Out)** principle, meaning the first element added is the first one removed.

Operations:

- **enqueue(x):** Adds element at the rear → **O(1)**
- **dequeue():** Removes element from the front → **O(1)**
- **front():** Returns front element → **O(1)**

☐ Efficiency is achieved by maintaining both **head and tail pointers**.

☐ 4. Explanation of Concepts

☐ **Dynamic Array**

- Automatically increases size when full
- Provides fast access and append operations
- Used in real-life like Python lists

☐ **Linked List**

- Stores elements dynamically using nodes
- No shifting required during insertion/deletion
- More flexible than arrays

☐ **Stack**

- Works on **LIFO principle**
- Used in:
 - Undo operations
 - Function calls
 - Expression evaluation

☐ **Queue**

- Works on **FIFO principle**

- Used in:
 - Task scheduling
 - Print queue
 - BFS traversal

□ Parentheses Checker (Using Stack)

The balanced parentheses problem checks whether an expression has properly matched brackets.

Rules:

- Every opening bracket must have a matching closing bracket
- Order must

be correct Example:

- " ([]) " → Balanced ✓
- " ([]] " → Not Balanced ✗

□ This is solved using a **stack**, where:

- Opening brackets are pushed
- Closing brackets are matched with the top element